Beyond Working Code

Applying SOLID Principles to Build Scalable & Maintainable Architecture

THE FOUNDATION

Defining SOLID

SOLID is an acronym for five design principles intended to make software designs more understandable, flexible, and maintainable.

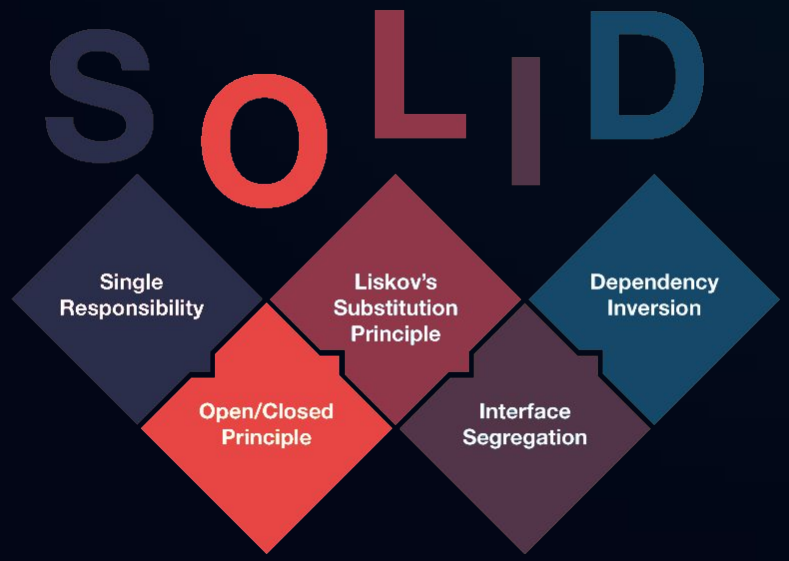
S - Single Responsibility

O - Open/Closed

L - Liskov Substitution

I - Interface Segregation

D - Dependency Inversion



THE PROBLEM: ARCHITECTURE EROSION

"The Messy Reality"

In real projects, as we keep adding features, code becomes messy. This is often caused by **tight coupling** and **lack of cohesion**.

"The Fragility Effect"

Small changes start breaking other parts. This happens because of poor design. Every new feature feels like a risk rather than progress.

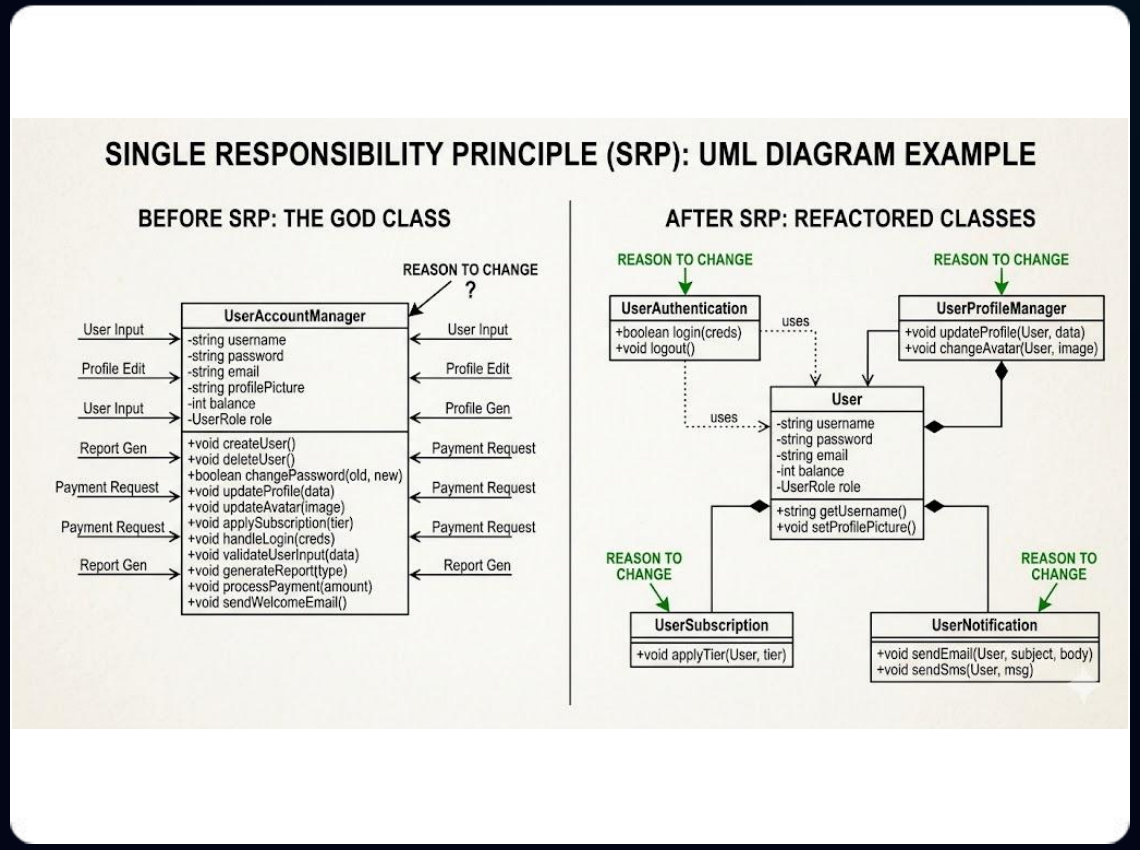
"So how do we solve this?"

S: SINGLE RESPONSIBILITY (SRP)

The Principle

SRP means one class should have **only one job**. If one class handles multiple unrelated tasks, it becomes hard to maintain.

> Goal: Avoid "God Classes" that handle data, logging, and storage simultaneously.



REFACTORING FOR SRP

User Logic

Handles core user authentication and state management.

Email Service

Isolated logic for notification delivery. Changing email logic no longer breaks user logic.

Data Persistence

Handles the database storage layer independently of business rules.

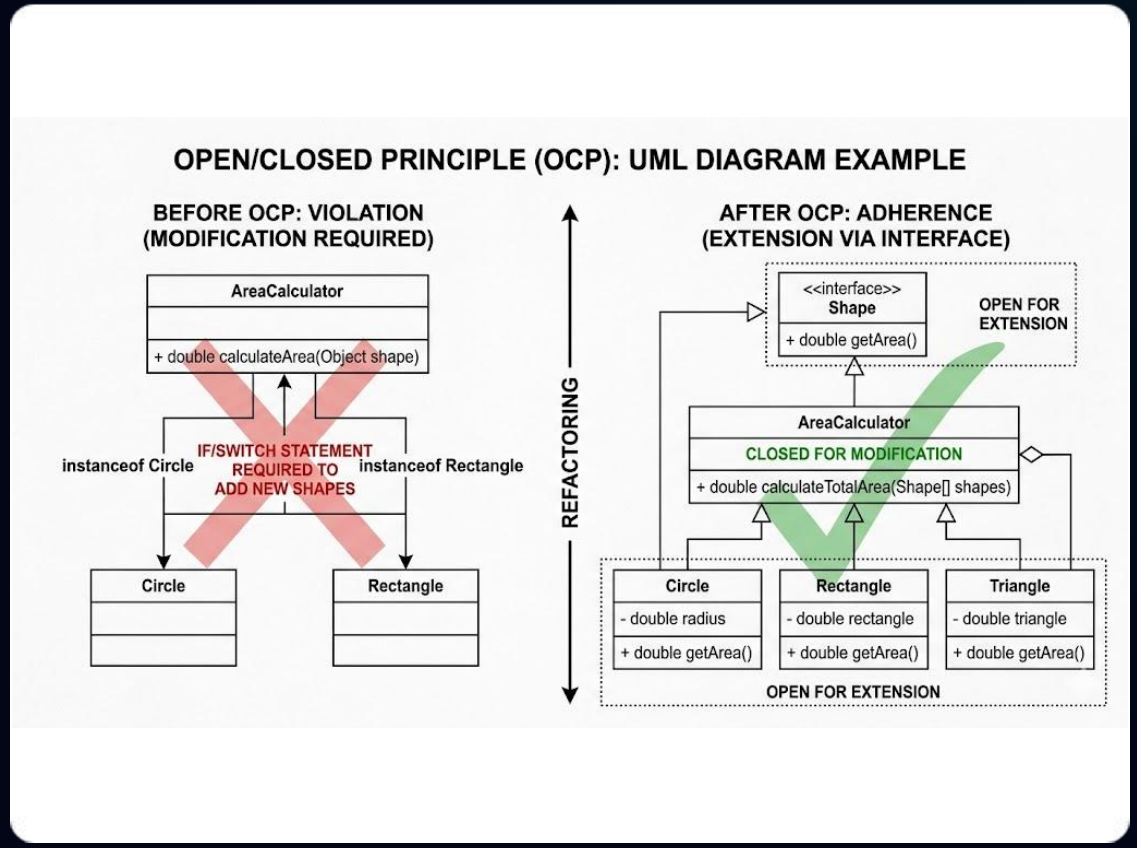
O: OPEN/CLOSED PRINCIPLE (OCP)

"Open for addition, closed for modification"

We should be able to add new features without modifying existing code.

Before: A violation forces us to rewrite AreaCalculator every time a new shape is added.

After: Use an <>. Adding 100 new shapes never changes the calculator.



REAL-WORLD APPLICATIONS

Banking Systems

Adding new transaction types or currency handlers without touching core ledger code.

E-Commerce

Plugging in new payment gateways (Stripe, PayPal, Crypto) using standard interfaces.

Microservices

Building highly modular APIs that scale independently and stay testable.

Thank You!

"Good code is not just working code — it's maintainable code."

>_ Questions & Discussion